

Laporan Praktikum Kontrol Cerdas Week 4

Nama : Bintang Ramadhan
NIM : 224308079
Kelas : TKA 7D
Akun Github (Tautan) : <https://github.com/bintangramadhan10>
Students Lab Assistant : -

1. Judul Laporan

Reinforcement Learning for Autonomous Control.

2. Tujuan Percobaan

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.

3. Landasan Teori

Reinforcement Learning (RL) merupakan salah satu metode pembelajaran mesin di mana agen belajar berinteraksi dengan lingkungan melalui proses trial and error. Agen menerima keadaan (state), memilih aksi (action), dan memperoleh balikan berupa reward. Tujuan utama dari RL adalah menemukan kebijakan (policy) yang memaksimalkan akumulasi reward jangka panjang. Berbeda dengan supervised learning yang membutuhkan label sebagai acuan, dalam RL agen harus belajar sendiri melalui eksplorasi dan evaluasi hasil tindakannya (Richard, n.d.).

Salah satu algoritma paling dasar dalam RL adalah Q-Learning yang menggunakan fungsi nilai aksi (action-value function) untuk menghitung ekspektasi reward kumulatif dari suatu pasangan state dan action. Namun,

metode Q-Learning tradisional menggunakan tabel Q (Q-Table) sehingga tidak efisien untuk state space yang besar dan kontinu. Untuk mengatasi hal tersebut, Deep Q-Network (DQN) diperkenalkan dengan memanfaatkan jaringan saraf tiruan sebagai function approximator bagi fungsi Q. Dengan pendekatan ini, agen dapat mempelajari hubungan kompleks antar state dan action meskipun lingkungannya memiliki dimensi tinggi (Sewak, 2019) .

Algoritma DQN pertama kali diperkenalkan oleh Volodymyr Mnih dan tim peneliti dari DeepMind pada tahun 2013 dalam makalah berjudul *Playing Atari with Deep Reinforcement Learning*. Dalam penelitian tersebut, agen berhasil memainkan permainan Atari hanya dengan mengandalkan input berupa piksel layar permainan. Keberhasilan ini disebabkan oleh dua teknik utama dalam DQN, yaitu experience replay dan target network. Experience replay menyimpan pengalaman agen dalam buffer sehingga pelatihan jaringan dilakukan dengan sampling acak, yang mengurangi korelasi antar data dan meningkatkan stabilitas. Sementara itu, target network digunakan untuk menghitung target nilai Q dan diperbarui secara berkala, sehingga mengurangi osilasi dan membuat pelatihan lebih stabil (Mnih et al., 2013).

Meski sukses secara empiris, algoritma DQN juga memiliki sejumlah keterbatasan, seperti kecenderungan menghasilkan estimasi berlebih (overestimation bias) pada nilai Q. Untuk mengatasi hal ini, berbagai modifikasi DQN dikembangkan, antara lain Double DQN yang memisahkan proses pemilihan aksi dan evaluasi aksi untuk mengurangi bias, serta Dueling DQN yang memisahkan representasi nilai state (value function) dan keunggulan aksi (advantage function) agar jaringan dapat menilai pentingnya suatu state meskipun tanpa informasi aksi tertentu. Variasi lain seperti Prioritized Experience Replay juga ditambahkan untuk menekankan pengalaman yang lebih signifikan dalam proses pembelajaran (Sewak, 2019).

4. Analisis dan Diskusi

Pada percobaan ini awalnya digunakan framework TensorFlow sesuai dengan modul praktikum. Namun, pada saat dijalankan di perangkat dengan Python 3.11, muncul error berupa kegagalan DLL load pada library internal TensorFlow. Hal ini terjadi karena versi TensorFlow yang tersedia belum sepenuhnya mendukung Python 3.11 dan sering bermasalah di sistem operasi Windows, sehingga pelatihan agen tidak dapat dilanjutkan. Sebagai solusi, dipilih framework PyTorch yang lebih fleksibel dan stabil untuk dijalankan pada Windows. PyTorch memiliki API yang sederhana, proses debugging lebih mudah, serta lebih ringan dijalankan dibanding TensorFlow. Dengan menggunakan PyTorch, pelatihan agen DQN dapat dilakukan tanpa kendala teknis, dan bahkan menunjukkan hasil yang sangat baik dengan jumlah episode yang lebih sedikit.

Hasil pelatihan memperlihatkan bahwa pada awal episode (0–100), skor agen masih rendah di bawah 50 karena agen berada pada fase eksplorasi penuh dengan $\epsilon = 1.0$. Seiring berjalannya pelatihan, performa meningkat secara bertahap hingga skor melampaui 200 pada pertengahan episode (150–200). Setelah episode ke-250, agen sudah mampu mencapai skor mendekati maksimum, yaitu 499. Hal ini membuktikan bahwa agen berhasil mempelajari strategi optimal dalam menjaga keseimbangan tiang. Pada tahap pengujian sebanyak 5 episode tanpa eksplorasi, agen secara konsisten memperoleh skor 499/500, yang menunjukkan bahwa agen benar-benar telah konvergen. Parameter yang digunakan berperan penting terhadap kinerja agen. Nilai γ tinggi (0.95–0.99) membuat agen memperhitungkan reward jangka panjang sehingga strategi lebih stabil. Mekanisme ϵ decay dari 1.0 menuju 0.22 memungkinkan agen menjaga keseimbangan antara eksplorasi di awal dan eksploitasi di akhir. Sementara itu, learning rate sebesar 0.001 memberikan stabilitas dalam pembaruan bobot jaringan; jika terlalu besar, pelatihan bisa menjadi tidak stabil, sedangkan jika terlalu kecil, proses belajar menjadi sangat lambat.

Meskipun tren skor meningkat, masih terdapat fluktuasi reward pada beberapa episode akibat agen melakukan eksplorasi. Agen juga memerlukan jumlah episode yang cukup besar untuk mencapai kestabilan performa, terbukti bahwa hasil mendekati maksimum baru diperoleh setelah lebih dari 250 episode. Secara konsep, Reinforcement Learning (RL) berbeda dengan Supervised Learning. RL tidak memerlukan dataset berlabel, melainkan belajar langsung dari interaksi dengan environment menggunakan reward sebagai umpan balik. Pendekatan ini lebih cocok untuk masalah kontrol dinamis seperti CartPole. Keseimbangan eksplorasi dan eksploitasi juga menjadi aspek kunci dalam RL. Eksplorasi memberi kesempatan agen menemukan strategi baru, sedangkan eksploitasi membuat agen memanfaatkan strategi terbaik yang sudah diketahui. Dalam percobaan ini, mekanisme epsilon decay terbukti efektif dalam menyeimbangkan kedua strategi tersebut. Potensi penerapan RL pada sistem kendali nyata sangat luas, seperti pada robotika (navigasi dan kendali lengan robot), kendaraan otonom (kontrol arah dan kecepatan), sistem energi (optimasi distribusi listrik), serta industri manufaktur (kontrol proses produksi otomatis). Dengan hasil yang dicapai, penggunaan PyTorch sebagai alternatif dari TensorFlow terbukti mampu memberikan solusi praktis dan efektif untuk implementasi RL dalam konteks pembelajaran kontrol otonom.

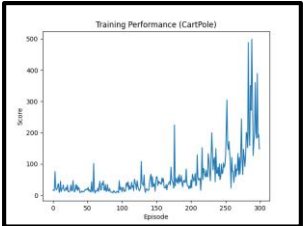
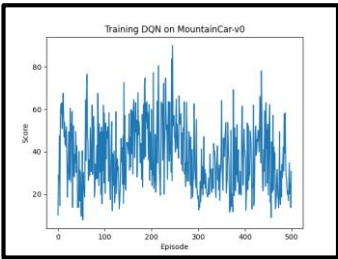
5. Assignment

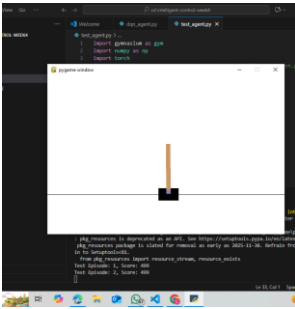
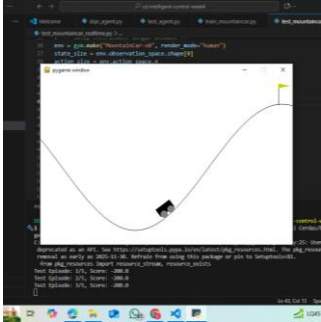
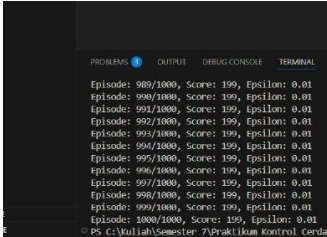
Pada percobaan ini, algoritma Deep Q-Network (DQN) telah dimodifikasi dan diuji pada environment MountainCar-v0. Hasil pengujian awal dengan 500 episode training menunjukkan bahwa performa agen masih tergolong lemah, ditunjukkan dengan skor konstan -200 pada 50 episode pengujian. Nilai -200 ini berarti agen selalu gagal mencapai target (bendera di puncak bukit) hingga batas langkah maksimum yang diizinkan oleh environment tercapai. Kondisi ini menggambarkan bahwa agen belum mampu menemukan strategi yang efektif untuk keluar dari lembah. Hal ini dapat terjadi karena beberapa faktor: kompleksitas lingkungan

MountainCar yang lebih tinggi dibandingkan CartPole, kurangnya eksplorasi yang optimal, serta keterbatasan jumlah episode training.

Namun, setelah dilakukan percobaan lanjutan dengan 1000 episode training, performa agen mengalami peningkatan signifikan. Pada percobaan ini diperoleh nilai/score puncak sebesar 199, yang menunjukkan bahwa agen mulai mampu menemukan strategi mendaki bukit menuju target. Hasil ini membuktikan bahwa jumlah episode training yang lebih besar mampu memberikan waktu belajar lebih banyak bagi agen sehingga jaringan saraf dapat menyesuaikan bobotnya secara lebih optimal.

6. Data dan Output Hasil Pengamatan

No	Perbandingan	CartPole-v1	MountainCar-v0
1.	Jumlah Episode Training	hanya dengan 300 episode, agen sudah mampu mencapai skor maksimum 499 dan stabil.	meskipun sudah dilatih 500 episode, agen tidak menunjukkan peningkatan signifikan, tetap gagal dengan skor -200.
2.	Performa Training	kor awal hanya 15–30 poin, namun meningkat cepat hingga >300 di pertengahan episode dan akhirnya mencapai 499 di akhir episode. 	skor training stagnan di sekitar -200 hingga -100, dengan sedikit fluktuasi namun tidak pernah mencapai skor positif. 

3.	Hasil Testing	hasil uji 5 episode konsisten dengan skor 499 (maksimal), artinya agen sudah sempurna menjaga keseimbangan tiang. 	hasil uji 50 episode semuanya -200, artinya agen tidak pernah berhasil mencapai target bendera di puncak bukit.  Setelah itu saya mencoba lagi training sejumlah 1000 epoc dan didapatkan nilai/score puncak 199. 
4.	Parameter yang Berpengaruh	Cartpole mempunyai Learning rate yang kecil, masih cukup untuk mendapat score baik.	Sedangkan untuk mountain car, Membutuhkan waktu lebih lambat agar eksplorasi lebih lama. membutuhkan episode lebih banyak ($\geq 1000-2000$) supaya agen sempat mencoba berbagai strategi. Parameter sama yang efektif di CartPole

			tidak langsung berhasil di MountainCar.
5.	Tantangan Utama	Tantangan hanya di awal training (tiang mudah jatuh sebelum agen belajar).	Tantangan besar sepanjang training karena reward yang jarang (sparse reward). Agen sulit belajar strategi karena sebagian besar aksinya tidak langsung menghasilkan reward positif.

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan dapat disimpulkan bahwa:

- Implementasi algoritma Deep Q-Network (DQN) pada environment CartPole-v1 berhasil dengan baik, ditunjukkan dengan skor maksimum 499 yang konsisten pada 5 episode pengujian.
- Pengujian awal pada environment MountainCar-v0 dengan 500 episode training menunjukkan performa agen masih lemah dengan skor konstan -200 pada pengujian, yang berarti agen gagal mencapai tujuan.
- Namun, setelah dilakukan training lanjutan sebanyak 1000 episode, performa agen meningkat signifikan dengan skor puncak 199, menunjukkan bahwa agen mulai mampu mempelajari strategi untuk mencapai target.

- Perbedaan performa ini dipengaruhi oleh tingkat kesulitan environment, di mana CartPole-v1 relatif sederhana, sedangkan MountainCar-v0 lebih kompleks dan membutuhkan strategi yang lebih lama dipelajari.
- Jumlah episode training yang terbatas akibat keterbatasan perangkat berpengaruh besar terhadap hasil pembelajaran, sehingga percobaan awal masih jauh dari optimal.

8. Saran

Untuk percobaan berikutnya, sebaiknya jumlah episode training diperpanjang hingga 1000 episode atau lebih agar agen memiliki waktu belajar yang cukup untuk menemukan strategi yang lebih baik. Penyesuaian parameter seperti gamma, epsilon, dan learning rate juga penting dilakukan supaya pembelajaran lebih stabil dan efisien. Selain itu, penggunaan perangkat dengan spesifikasi lebih tinggi akan sangat membantu mempercepat proses training yang relatif lama pada environment kompleks seperti MountainCar. Uji coba pada environment lain, misalnya LunarLander-v2, juga disarankan agar hasil pembelajaran DQN bisa dibandingkan pada kasus yang lebih menantang sekaligus memperkaya analisis performa algoritma.

9. Daftar Pustaka

- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). *Playing Atari with Deep Reinforcement Learning* (No. arXiv:1312.5602). arXiv. <https://doi.org/10.48550/arXiv.1312.5602>
- Richard, S. S. (n.d.). *Reinforcement Learning: An Introduction*. MIT Press.
- Sewak, M. (2019). Deep Q Network (DQN), Double DQN, and Dueling DQN: A Step Towards General Artificial Intelligence. In M. Sewak, *Deep Reinforcement Learning* (pp. 95–108). Springer Singapore. https://doi.org/10.1007/978-981-13-8285-7_8